

Operating System (ISE3137)

Jump Together, Fly Farther!

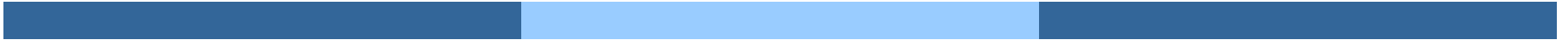


인하대학교 국제학부

Week 9 Lecture

**ISE Department
Prof. Mehdi Pirahandeh**

Chapter 7: Synchronization Examples



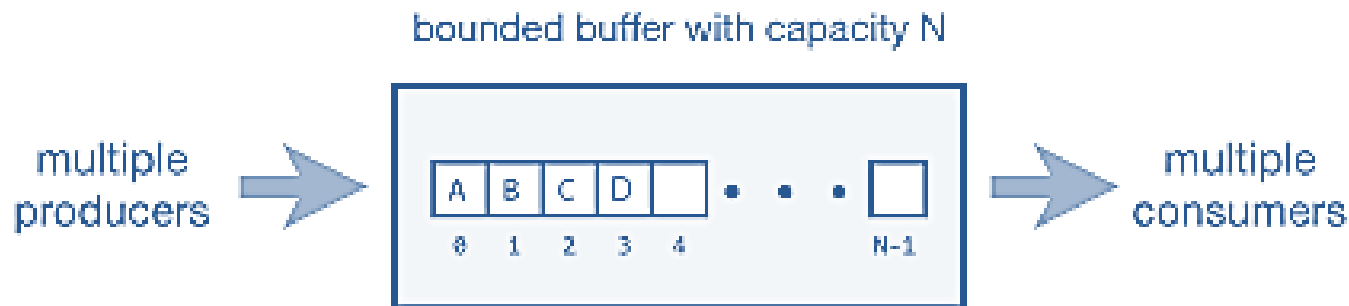
Classical Problems of Synchronization



- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem

Bounded-Buffer Problem

- n buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n



Bounded Buffer Problem (Cont.)



인하대학교
INHA UNIVERSITY

Producer

```
do {  
    wait (empty); // wait until empty>0  
                    and then decrement 'empty'  
    wait (mutex); // acquire lock  
    /* add data to buffer */  
    signal (mutex); // release lock  
    signal (full); // increment 'full'  
} while(TRUE)
```

Consumer

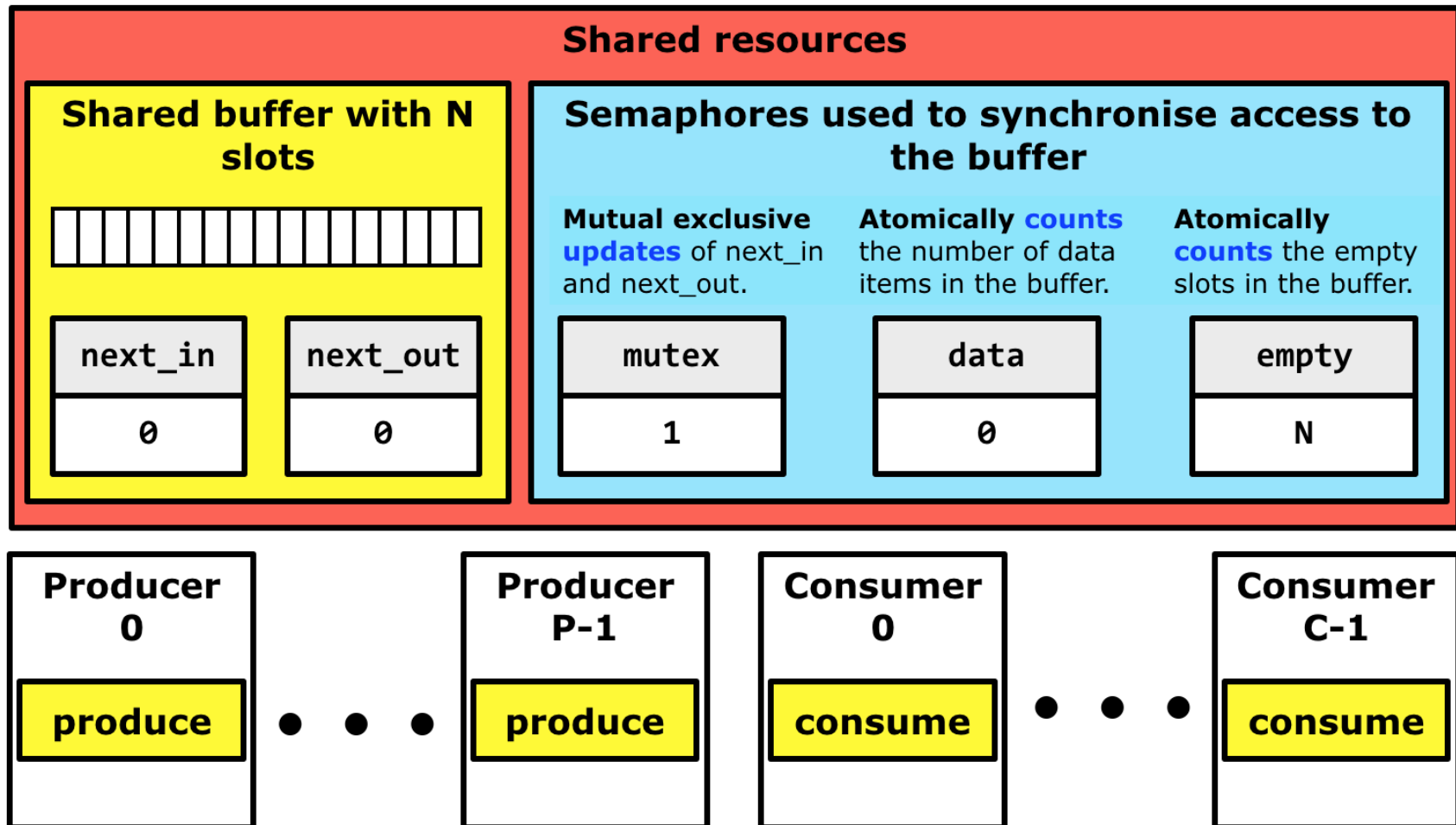
```
do {  
    wait (full); // wait until full>0 and  
                  then decrement 'full'  
    wait (mutex); // acquire lock  
    /* remove data from buffer */  
    signal (mutex); // release lock  
    signal (empty); // increment 'empty'  
} while(TRUE)
```

Bounded Buffer Problem (Cont.)



인하대학교
INHA UNIVERSITY

- [Reference link](#)

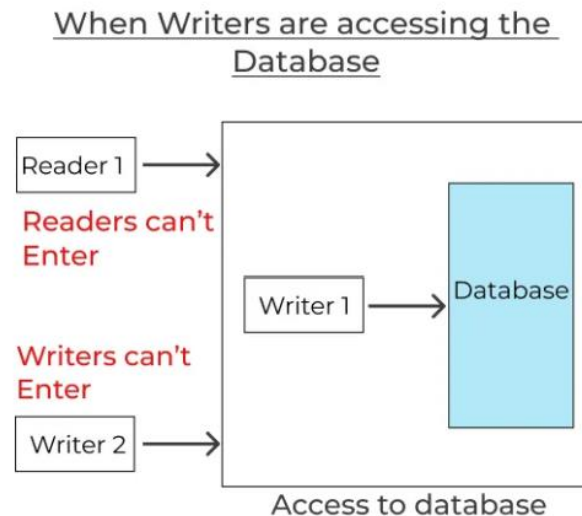
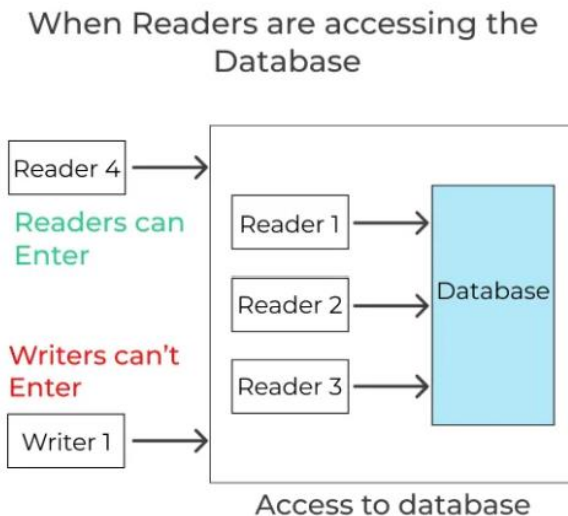


Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do **not** perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do **not** perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

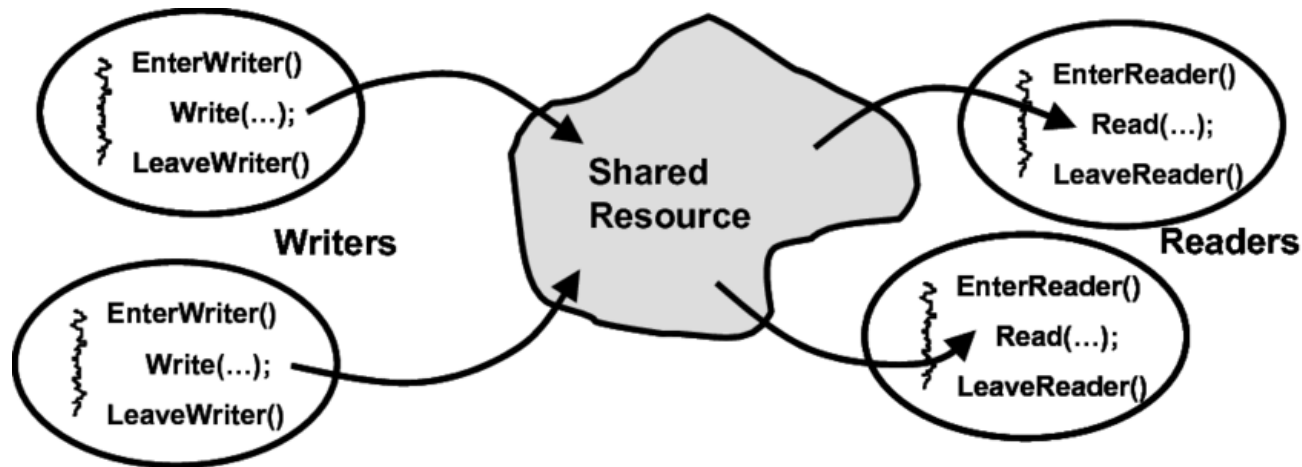


Readers-Writers Problem (Cont.)



인하대학교
INHA UNIVERSITY

- Shared Data
 - Data set
 - Semaphore **rw_mutex** initialized to 1
 - Semaphore **mutex** initialized to 1
 - Integer **read_count** initialized to 0



Readers-Writers Problem (Cont.)



인하대학교
INHA UNIVERSITY

Writer Process

```
do {  
  /* writer requests for critical  
  section */  
  wait(wrt);  
  /* performs the write */  
  // leaves the critical section  
  signal(wrt);  
} while(true);
```

Reader Process

```
do {  
  wait (mutex);  
  readcnt++; // The number of readers has now increased by 1  
  if (readcnt==1)  
    wait (wrt); // this ensure no writer can enter if there is even one reader  
  signal (mutex); // other readers can enter while this current reader is  
                  inside the critical section  
  /* current reader performs reading here */  
  wait (mutex);  
  readcnt--; // a reader wants to leave  
  if (readcnt == 0) //no reader is left in the critical section  
    signal (wrt); // writers can enter  
  signal (mutex); // reader leaves  
} while(true);
```

Readers-Writers Problem Variations

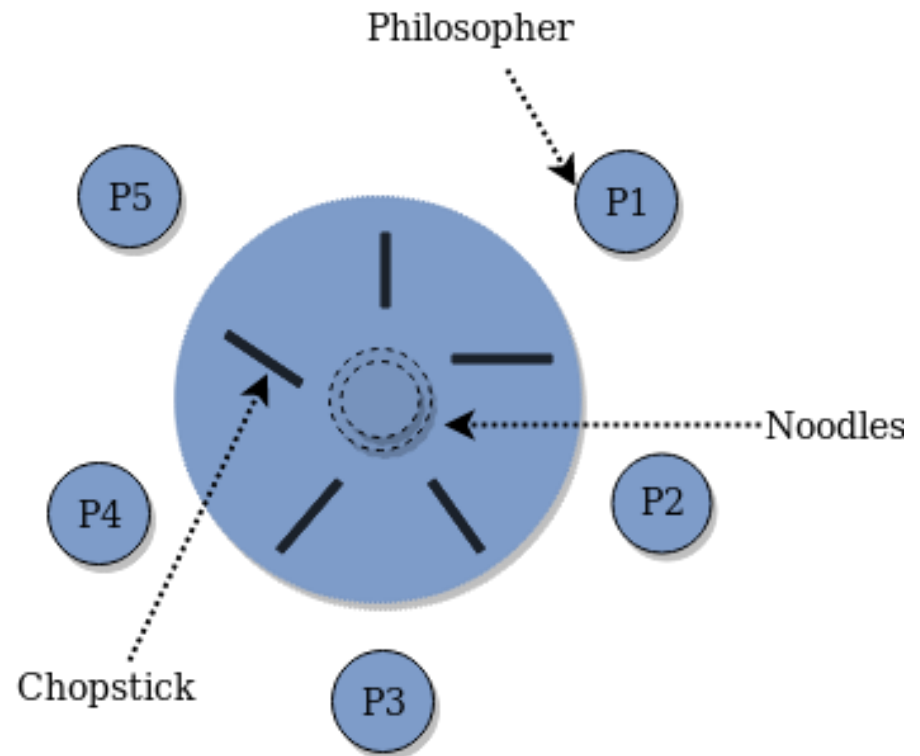


- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
 - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation. leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

Dining-Philosophers Problem Algorithm

- Philosophers either eat or think.

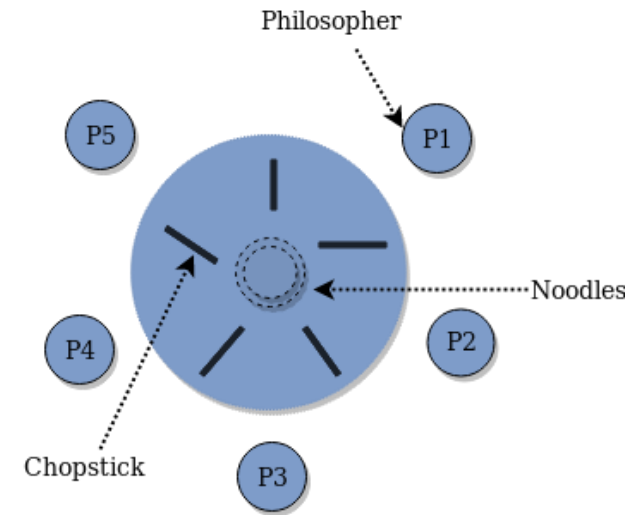
```
The structure of philosopher i
do {
    wait (chopstick [i] );
    wait(chopstick [ (i + 1) % 5] );
    ....
    // eat
    signal(chopstick [i]);
    signal(chopstick [(i + 1) % 5]);
    // think
}while (TRUE);
```



Dining-Philosophers Problem Algorithm

- There are three states of the philosopher: THINKING, HUNGRY, and EATING.
- Here there are two semaphores: Mutex and a semaphore array for the philosophers.
- Mutex is used such that no two philosophers may access the pickup or putdown at the same time.
- The array is used to control the behavior of each philosopher.
- But, semaphores can result in deadlock due to programming errors.

```
process P[i]
while true do
{  THINK;
  PICKUP(CHOPSTICK[i], CHOPSTICK[i+1 mod 5]);
  EAT;
  PUTDOWN(CHOPSTICK[i], CHOPSTICK[i+1 mod 5])
}
```



Linux Synchronization

- Atomic variables

`atomic_t` is the type for atomic integer

- Consider the variables

```
atomic_t counter;  
int value;
```

<i>Atomic Operation</i>	<i>Effect</i>
<code>atomic_set(&counter,5);</code>	<code>counter = 5</code>
<code>atomic_add(10,&counter);</code>	<code>counter = counter + 10</code>
<code>atomic_sub(4,&counter);</code>	<code>counter = counter - 4</code>
<code>atomic_inc(&counter);</code>	<code>counter = counter + 1</code>
<code>value = atomic_read(&counter);</code>	<code>value = 12</code>

POSIX Synchronization

- POSIX API provides
 - mutex locks
 - semaphores
 - condition variable
- Widely used on UNIX, Linux, and macOS

POSIX Mutex Locks

- Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

- Acquiring and releasing the lock

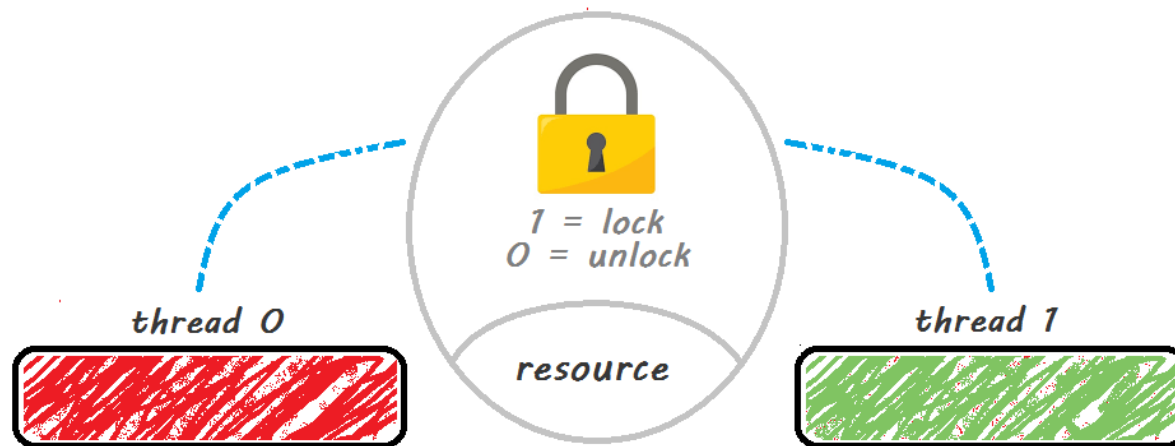
```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

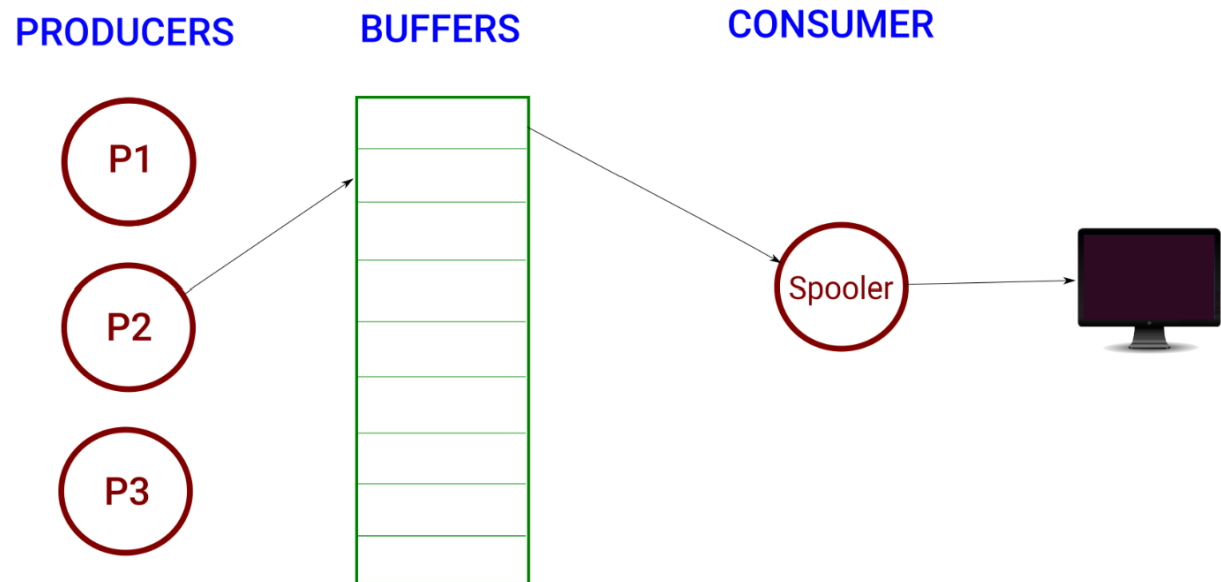
POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.



POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.
- In this example, the producers produce strings of text. The consumer thread prints the strings on the terminal.
- **Named Semaphores**



POSIX Named Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

POSIX Unnamed Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;  
pthread_cond_t cond_var;  
  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cond_var, NULL);
```

POSIX Condition Variables

- Thread waiting for the condition **a == b** to become true:

```
pthread_mutex_lock(&mutex);  
while (a != b)  
    pthread_cond_wait(&cond_var, &mutex);  
  
pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);  
a = b;  
pthread_cond_signal(&cond_var);  
pthread_mutex_unlock(&mutex);
```

Alternative Approaches

- Transactional Memory
- OpenMP
- Functional Programming Languages

Transactional Memory

- Consider a function `update()` that must be called atomically. One option is to use mutex locks:

```
void update ()
{
    acquire();

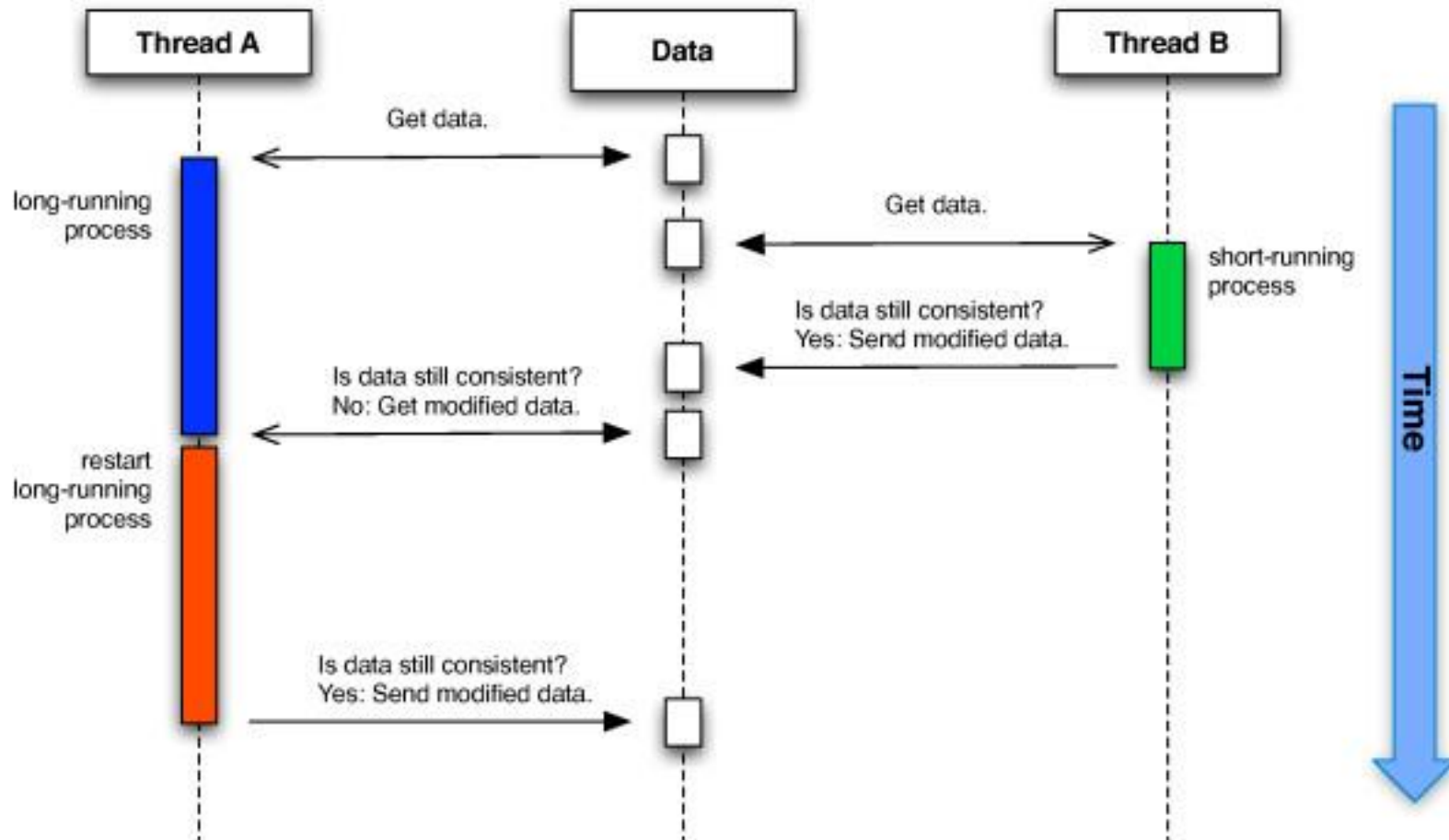
    /* modify shared data */

    release();
}
```

- A **memory transaction** is a sequence of read-write operations to memory that are performed atomically. A transaction can be completed by adding **`atomic{S}`** which ensure statements in **`S`** are executed atomically:

```
void update ()
{
    atomic {
        /* modify shared data */
    }
}
```

Transactional Memory



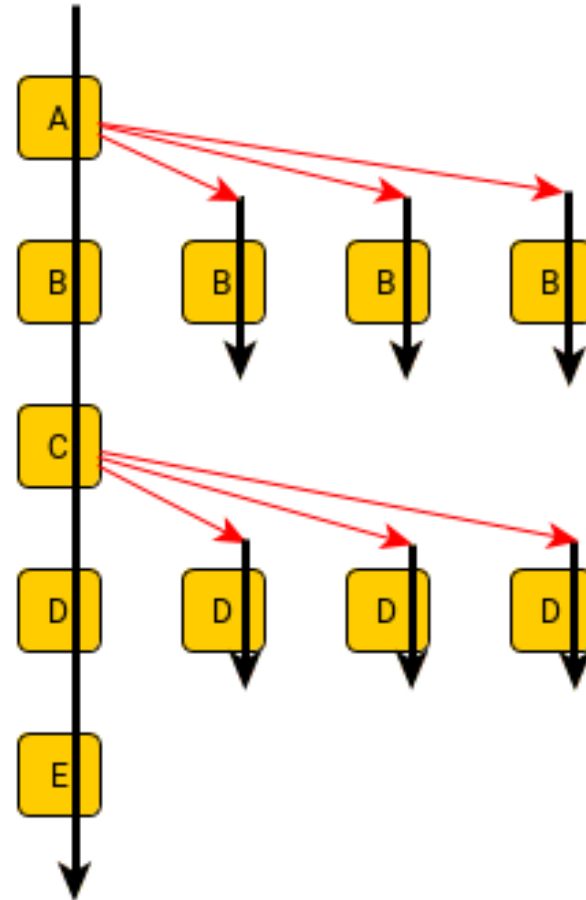
- OpenMP is a set of compiler directives and API that support parallel programming.

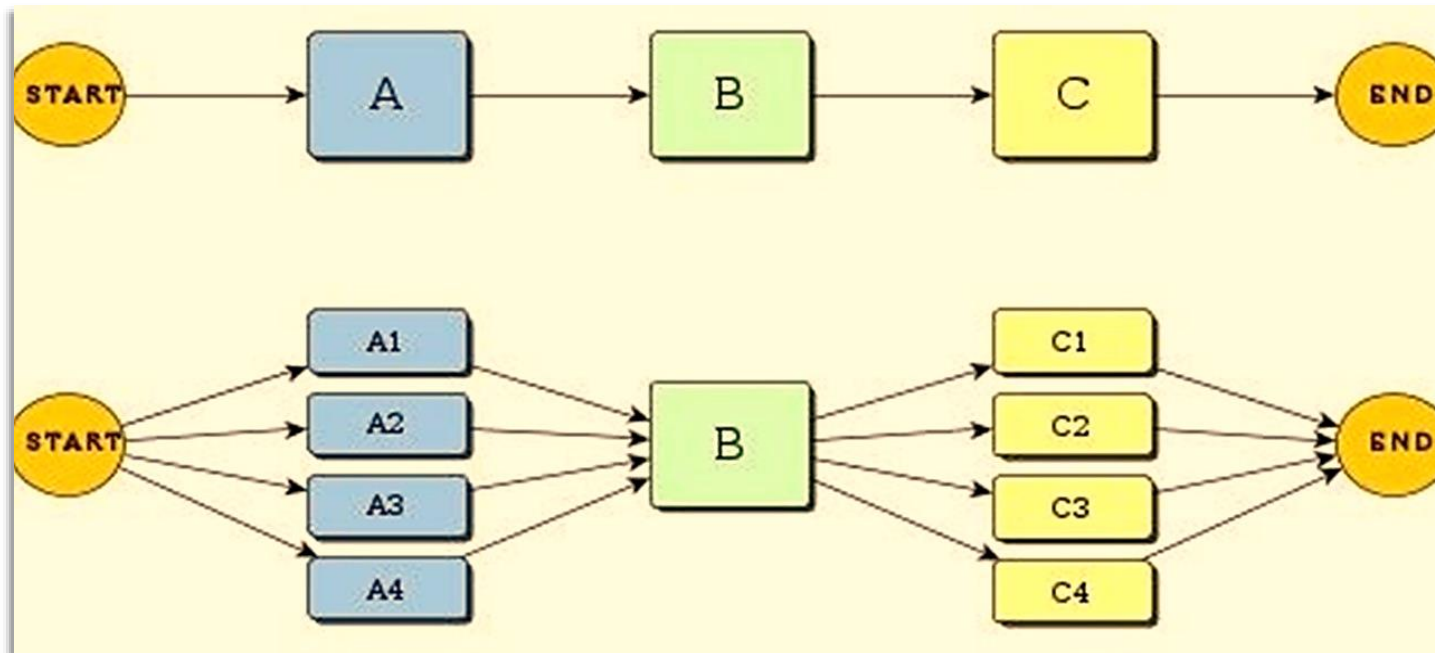
```
void update(int value)
{
    #pragma omp critical
    {
        count += value
    }
}
```

- The code contained within the **#pragma omp critical** directive is treated as a critical section and performed atomically.

```
#include "omp.h"

int main() {
    // A - single thread
    #pragma omp parallel
    {
        // B - many threads
    }
    // C - single thread
    #pragma omp parallel
    {
        // D - many threads
    }
    // E - single thread
}
```





Functional Programming Languages

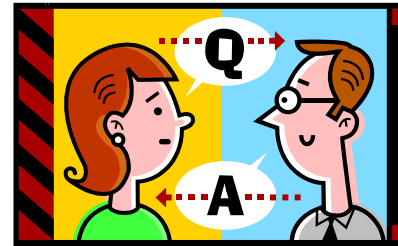


인하대학교
INHA UNIVERSITY

- Functional programming languages offer a different paradigm than procedural languages in that they do not maintain state.
- Variables are treated as immutable and cannot change state once they have been assigned a value.
- There is increasing interest in functional languages such as Erlang and Scala for their approach in handling data races.

Brief break (if on schedule)

Q&A?



Prof. Mehdi Pirahandeh
E-mail : mehdi@inha.ac.kr